Extended Abstract

Motivation Advancements in Vision Language Action models (VLA's) in the last year have been the backbone of many advancements in robotics and AI. As model architectures continue to improve, the bottleneck shifts from training to inference performance, as robots need to process visual and textual information about their environment quickly in order to make quick decisions. Unlike text based transformers, a 1 percent decrease in robotic task accuracy can lead to financial costs and human injuries. Recent work this year has applied quantization techniques to LLM's and more recently VLA's. Many of these approaches tend to apply quantization uniformly to all layers of the VLA. Furthermore, the weights of linear layers inside the action expert's MLP blocks are notoriously sensitive to quantization.

Method We study post-training quantization of the action-generation components in $\pi 0.5$ and Nvidia's GR00T N1.7. We evaluate FP16, W4A4, W4A8, and activation-only A8 configurations using symmetric uniform quantization. Each tensor is mapped to a signed integer grid using a maximum-absolute-value scale and then de-quantized before multiplication, reproducing quantization error without requiring retraining. By default, quantization is restricted to the action-expert MLPs, leaving attention and vision-language components in full precision. We evaluate each configuration on the four LIBERO suites using task success rate and GPU memory measurements. We also conduct a separate low-bit attention study using a Walsh-Hadamard transform followed by a fixed Lloyd-Max codebook to analyze K/V activation drift across denoising steps.

Implementation The quantization hooks were integrated directly into the native inference code of both models. For $\pi 0.5$, implemented in JAX/OpenPI, we quantize the gating and output projections of the action-expert MLPs in all 18 transformer blocks. For GR00T N1.7, implemented in PyTorch, we wrap the linear layers of the DiT feed-forward networks in all 12 transformer blocks. GR00T uses per-token activation scales and per-output-channel weight scales, while the 0.5 implementation uses tensor-level scaling. An optional full-quantization mode additionally quantizes attention-related activations and projections. The implementation also supports an experimental 3-state ITQ3S weight reconstruction based on Hadamard rotation and ternary values, although the main evaluation matrix uses uniform W4A4, W4A8, and A8 quantization. Experiments run through each model's native LIBERO evaluation pipeline on Modal, with automated collection of success rates, inference timing, and GPU memory.

Results Across the LIBERO tests, most of the success rates remained about the same. More aggressive W4A4 quantization reduced average success especially for GR00T 1.7. WHT-Lloyd-Max analysis produced low live quantization MSE for $\pi 0.5$ K/V tensors (0.009–0.013) and moderate error for GR00T cross-attention tensors (approximately 0.051–0.052). GPU memory remained effectively unchanged across presets, averaging approximately 30.57 GB for 0.5 and 7.16 GB for GR00T, because the implementation simulates quantization numerically without packed low-bit weight storage. For our low bit attention study, we found that trying to quantize below 4 bits for the action weights led to severe accuracy degradation across all the LIBERO tasks.

Discussion These results indicate that moderate post-training quantization can preserve VLA task performance, but robustness depends strongly on precision and task complexity. A8 was the most reliable configuration, matching or nearly matching FP16 on both models. W4A8 also retained performance whereas W4A4 and anything quantized beyond that degraded performance. Its disproportionate failure on LIBERO-Long suggests that accumulated quantization error is more damaging for tasks requiring extended, multi-stage control.

Conclusion This work evaluated the quantization sensitivity of $\pi_0.5$ and GR00T N1.7 across four LIBERO task suites. A8 and W4A8 largely preserved FP16 success, whereas W4A4 caused substantial degradation, particularly on object-centric and long-horizon tasks. K/V drift and distribution analyses further showed that low average quantization error does not always guarantee closed-loop robustness. Because our implementation uses fake quantization, it evaluates numerical sensitivity without realizing low-bit memory compression. Overall, the results identify W4A8 as a promising accuracy-preserving configuration and motivate future work using calibrated quantization, packed weights, and hardware-supported low-bit kernels.

Compressing Vision-Language-Action Models via Backbone and DiT Quantization

William Yang
Department of Computer Science
Stanford University
yangwa@stanford.edu

Abstract

Vision-language-action (VLA) models achieve strong robotic manipulation performance but impose substantial computational and memory costs. This work evaluates the robustness of 0.5 and GR00T N1.7 to simulated post-training quantization on the LIBERO benchmark. We compare FP16 inference with activation-only 8-bit quantization (A8), 4-bit weight and 8-bit activation quantization (W4A8), and 4-bit weight and activation quantization (W4A4), targeting the action-expert and DiT MLP layers. Across four LIBERO suites, FP16 achieved average success rates of 97.63% for π0.5 and 98.50% for GR00T N1.7. A8 largely preserved performance at 96.88% and 98.85%, while W4A8 achieved 95.88% and 97.51%, respectively. In contrast, W4A4 reduced success to 76.25% for π0.5 and 83.00% for GR00T, with the largest degradation on object-centric and long-horizon tasks. Tensor-level drift and distribution analyses show that moderate quantization preserves K/V activation structure with relatively low live quantization error, whereas aggressive quantization introduces discrete information loss that disproportionately affects extended control sequences. GPU memory remained approximately constant across presets because the implementation uses quantization while retaining floating-point parameter storage. These results suggest that A8 and W4A8 offer favorable accuracy trade-offs for VLA inference, while W4A4 requires improved calibration or quantization-aware methods. They also demonstrate that behavioral evaluation and tensor-level diagnostics are both necessary when assessing quantized VLA policies.

1 Introduction

Vision-language-action (VLA) models have emerged as a promising approach to general-purpose robotic control. By combining visual observations, language instructions, and learned action generation within a single policy, these models can transfer semantic knowledge from large-scale vision-language pretraining to diverse manipulation tasks. Recent systems such as 0.5 and GR00T N1.7 demonstrate strong performance across multi-task robotic benchmarks, but their large parameter counts and computational requirements make deployment on memory and power-constrained robotic platforms difficult. Furthermore, the action head in such models is incredibly sensitive to deviations in quantization errors that compound quickly Li et al. (2026).

Post-training quantization offers a potential path toward more efficient VLA inference by representing model weights and activations at reduced numerical precision. In February of 2026, the first quantization approach for VLA's was published where they integrated two paradigms, attention temperature matching and output head balancing, into both the LLM backbone and DiT action head Zhang et al. (2026).

This work studies the quantization sensitivity of π0.5 and GR00T N1.7 on the LIBERO benchmark. We evaluate four numerical configurations: FP16, activation-only 8-bit quantization (A8), 4-bit weights with 8-bit activations (W4A8), and 4-bit weights with 4-bit activations (W4A4). Quantization is applied to the action-expert or diffusion-transformer MLP layers while leaving attention operations unquantized. We measure success across the LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-Long suites, which collectively test spatial reasoning, object manipulation, goal-conditioned behavior, and long-horizon control.

In addition to task success, we analyze GPU memory usage, activation distributions, quantization error, and feature drift across denoising steps. These measurements help distinguish behavioral degradation from tensor-level distortion and reveal limitations that aggregate success rates alone may conceal. In particular, the K/V distributions exhibit model-dependent behavior: π0.5 contains heavy-tailed activations, whereas quantized GR00T representations concentrate at discrete levels.

Our results show that moderate quantization preserves most of the original policy performance. A8 remains within 0.75 percentage points of FP16 for π0.5 and slightly exceeds the measured GR00T baseline, while W4A8 remains within 1.75 and 0.99 percentage points, respectively. In contrast, W4A4 causes substantial degradation, particularly on object-centric and long-horizon tasks.

2 Related Work

Efforts to quantize the expensive matrix operations in transformer architectures have made significant progress. Our work builds on the efforts of QuantVLA Zhang et al. (2026) which originally uses a temperature matching and head approach to apply quantization to both the VLM backbone and DiT action head of Gr00t 1.5 and π0.5 as well. Less than 3 months after the publication of QuantVLA, Google published TurboQuant, a quantization based algorithm that significantly leverages the existing Lindenburg-Strauss transform and polar quantization techniques to compress the KV cache matrix into polar coordinate form, slashing memory by over 5 times, and reducing floating point representation to 3 bits Zandieh et al. (2026).

Post-training quantization (PTQ) reduces inference cost by mapping floating-point weights and activations to lower-precision representations without retraining the original model. quantization is commonly expressed as

$$s_x = \frac{\max |x|}{2^{b-1}-1}, \qquad Q_b(x) = s_x \,\mathrm{clip}\left(\mathrm{round}\left(\frac{x}{s_x}\right), -2^{b-1}, 2^{b-1}-1\right).$$

For a linear layer $y = Wx$, weight-activation quantization approximates the operation using $Q_{b_w}(W)Q_{b_a}(x)$. This notation produces configurations such as W4A4 and W4A8, where b_w and b_a denote weight and activation precision, respectively. A8 applies Q_8 only to activations while retaining floating-point weights, whereas FP16 provides the unquantized reference. LLM quantization research has shown that activation outliers are a central obstacle to low-bit inference. SmoothQuant migrates quantization difficulty from activations to weights through an equivalent channel-wise rescaling, enabling accurate W8A8 inference Xiao et al. (2022). AWQ instead uses activation statistics to identify and protect salient weight channels, supporting hardware-efficient low-bit weight compression Lin et al. (2024a). QServe extends this line to W4A8KV4 inference through progressive quantization, attention smoothing, and system-level kernel design Lin et al. (2024b). These methods establish W4A8 as an attractive compromise: four-bit weights reduce storage and bandwidth, while eight-bit activations retain substantially more dynamic range than W4A4. Rotation-based methods address outliers by redistributing their energy across channels. Given an orthogonal Walsh-Hadamard matrix H and a random sign matrix D, a vector can be transformed as

$$\widetilde{x} = \frac{1}{\sqrt{d}} HDx, \qquad \widehat{x} = \frac{1}{\sqrt{d}} DH^\top Q_b(\widetilde{x}).$$

Because the transformation is orthogonal, it preserves norms and inner products before quantization while making the coordinate distribution easier to represent with a low-bit scalar code. The Walsh-Hadamard matrix is also used in an extension of Turboquant which is defined as the following: The Walsh-Hadamard Transform of a sequence $x(n)$ of length $N = 2^m$ is defined via the Hadamard matrix H_N. The lowest-order Hadamard matrix is $H_1 = [1]$, and for $N \geq 2$, the matrix is defined recursively in natural (Kronecker) order as:

$$H_N = H_{2^m} = H_2 \otimes H_{2^{m-1}} = \begin{bmatrix} H_{2^{m-1}} & H_{2^{m-1}} \\ H_{2^{m-1}} & -H_{2^{m-1}} \end{bmatrix} \tag{1}$$

Where the base matrix H_2 is:

$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \tag{2}$$

For a column vector $\mathbf{x}$ of length N, the forward Walsh-Hadamard Transform vector $\mathbf{X}$ is computed as:

$$\mathbf{X} = H_N \mathbf{x} \tag{3}$$

The inverse transform is identical up to a scaling factor, exploiting the symmetric and orthogonal properties of the matrix ($H_N^2 = N I_N$):

$$\mathbf{x} = \frac{1}{N} H_N \mathbf{X} \tag{4}$$

This is followed up by the Lloyd Max transform that compresses the quantization into a fixed set of discrete values Lloyd (1982).

3 Method

3.1 Method Overview

Our study proceeded in two stages. We first measured the numerical behavior of attention key-value (K/V) tensors across action denoising events. We then shifted from quantizing the combined language-model and diffusion-transformer pipeline to selectively quantizing the action-generating components of each VLA. The final experiments targeted the $\pi_0.5$ action-expert MLPs and the GR00T N1.7 DiT feed-forward MLPs while retaining attention projections in floating point.

3.2 K/V Drift Analysis

We initially instrumented the models to record attention K/V tensors before and after quantization. For $\pi_0.5$, the hooks recorded K/V tensors from the action-conditioned Gemma attention path. For GR00T, they recorded key and value projections from a selected DiT cross-attention layer.

Before executing the instrumented policies in LIBERO, we constructed a synthetic sanity test with ten indexed events for the $\pi_0.5$ prefix and action tensors and four events for GR00T. This verified tensor collection, quantization, statistic computation, and visualization independently of the simulator.

For a tensor $\mathbf{x}_t \in \mathbb{R}^n$ recorded at event t, we measured its root-mean-square energy as

$$R_t = \mathrm{RMS}(\mathbf{x}_t) = \sqrt{\frac{1}{n} \sum_{i=1}^{n} x_{t,i}^2}. \tag{5}$$

We measured relative RMS drift from the first recorded event using

$$D_t = \frac{|R_t - R_0|}{\max(R_0, \epsilon)}, \qquad \epsilon = 10^{-8}. \tag{6}$$

For an original tensor $\mathbf{x}_t$ and its quantized reconstruction $\widehat{\mathbf{x}}_t$, quantization error was measured as

$$E_t = \mathrm{MSE}\left(\mathbf{x}_t, \widehat{\mathbf{x}}_t\right) = \frac{1}{n} \left\| \mathbf{x}_t - \widehat{\mathbf{x}}_t \right\|_2^2. \tag{7}$$

We compared three scale-selection settings: a static scale obtained from the first event, a scale recomputed independently at each event, and the actual live reconstruction generated by the runtime quantization hook.

We additionally measured the mean, standard deviation, and kurtosis of each tensor. Histograms were used to visualize whether the transformed and quantized distributions preserved the original K/V structure or collapsed onto a small number of reconstruction levels.

3.3 WHT–Lloyd–Max K/V Quantization

The initial K/V experiment used a normalized Walsh–Hadamard transform (WHT) followed by Lloyd–Max scalar quantization. For channel dimension d, the forward transform was

$$\mathbf{z}_t = \frac{1}{\sqrt{d}}\mathbf{H}\mathbf{x}_t, \tag{8}$$

where $\mathbf{H}$ is a Hadamard matrix satisfying $\mathbf{H}^\top\mathbf{H} = d\mathbf{I}$.

We normalized the transformed tensor using its RMS:

$$s_t = \sqrt{\frac{1}{n}\sum_{i=1}^{n} z_{t,i}^2}, \qquad \mathbf{u}_t = \frac{\mathbf{z}_t}{\max(s_t, \epsilon)}. \tag{9}$$

Each normalized coordinate was assigned to a centroid from a precomputed Lloyd–Max codebook:

$$Q_{\mathrm{LM}}(u) = c_j \quad \text{for} \quad \tau_{j-1} < u \le \tau_j, \tag{10}$$

where c_j is a reconstruction centroid and τ_j is the corresponding decision threshold.

The reconstructed tensor was

$$\widehat{\mathbf{x}}_t = \frac{1}{\sqrt{d}}\mathbf{H}^\top \left(s_t Q_{\mathrm{LM}}(\mathbf{u}_t)\right). \tag{11}$$

The primary diagnostic used a 3-bit codebook containing eight centroids. This stage evaluated K/V stability and did not define the final precision matrix used for LIBERO accuracy evaluation.

3.4 Action-Specific Quantization Scope

The K/V analysis characterized internal representation drift but did not directly isolate the source of closed-loop action failures. Quantizing the entire VLA would jointly perturb perception, language processing, attention, and action generation. We therefore restricted the final experiments to the action-generating MLP components.

For $\pi_0.5$, quantization was applied to the feed-forward modules of all 18 action-expert transformer blocks. Each gated feed-forward module computes

$$\mathbf{g} = \mathrm{GELU}\left(\mathbf{x}\mathbf{W}_g\right), \tag{12}$$

$$\mathbf{u} = \mathbf{x}\mathbf{W}_u, \tag{13}$$

$$\mathbf{h} = \mathbf{g} \odot \mathbf{u}, \tag{14}$$

$$\mathbf{y} = \mathbf{h}\mathbf{W}_o, \tag{15}$$

where $\odot$ denotes element-wise multiplication. Quantization was inserted before the two input projections and the output projection.

For GR00T N1.7, each linear layer in the DiT feed-forward network was wrapped with the quantization operator. This covered the DiT MLPs in all 12 transformer blocks. Under the selected scope, attention projections remained in their original precision.

This selective layout is denoted "without DiT attention." It was used for A8, W4A8, and W4A4. FP16 used the same architecture without quantization.

3.5 Uniform Weight and Activation Quantization

For a tensor $\mathbf{x}$ and target precision b, we used symmetric uniform quantization. The maximum signed integer value is

$$q_{\max} = 2^{b-1} - 1. \tag{16}$$

The scale is computed as

$$s(\mathbf{x}) = \frac{\max_i |x_i|}{\max(q_{\max}, 1)}. \tag{17}$$

The reconstructed tensor is

$$Q_b(\mathbf{x}) = s(\mathbf{x}) \cdot \mathrm{clip}\left(\mathrm{round}\left(\frac{\mathbf{x}}{\max(s(\mathbf{x}), \epsilon)}\right), -q_{\max}, q_{\max}\right). \tag{18}$$

The GR00T implementation used per-token activation scales and per-output-channel weight scales. The $\pi_0.5$ JAX implementation used tensor-level symmetric scales within its action-expert hooks.

For a linear operation $\mathbf{y} = \mathbf{x}\mathbf{W}$, the four evaluated settings were

$$\text{FP16}: \quad \widehat{\mathbf{x}} = \mathbf{x}, \qquad \widehat{\mathbf{W}} = \mathbf{W}, \tag{19}$$

$$\text{A8}: \quad \widehat{\mathbf{x}} = Q_8(\mathbf{x}), \qquad \widehat{\mathbf{W}} = \mathbf{W}, \tag{20}$$

$$\text{W4A8}: \quad \widehat{\mathbf{x}} = Q_8(\mathbf{x}), \qquad \widehat{\mathbf{W}} = Q_4(\mathbf{W}), \tag{21}$$

$$\text{W4A4}: \quad \widehat{\mathbf{x}} = Q_4(\mathbf{x}), \qquad \widehat{\mathbf{W}} = Q_4(\mathbf{W}). \tag{22}$$

The quantized linear output was then

$$\widehat{\mathbf{y}} = \widehat{\mathbf{x}}\widehat{\mathbf{W}} + \mathbf{b}. \tag{23}$$

No calibration training or quantization-aware fine-tuning was performed. The same precision was applied to every eligible action MLP layer.

3.6 Quantization Implementation

The implementation simulated quantization numerically. Weights and activations were mapped onto the requested low-bit grid and then reconstructed in their original floating-point type before the matrix multiplication. The original floating-point weights remained allocated throughout inference.

Consequently, the method evaluates the behavioral sensitivity of each policy to quantization error. It does not implement packed integer parameters, integer accumulation, or low-bit matrix-multiplication kernels. Memory results therefore describe the allocation behavior of the quantized prototype rather than the compression ratio of a deployment-ready quantized runtime.

3.7 ITQ3_S-Inspired W3A8 Attempt

Before selecting the final precision matrix, we implemented an exploratory W3A8 path inspired by ITQ3_S. A normalized WHT was first applied to each weight tensor:

$$\mathbf{Z} = \frac{1}{\sqrt{d}}\mathbf{H}\mathbf{W}. \tag{24}$$

The ternary reconstruction scale was estimated as

$$\alpha = \frac{2}{3}\,\mathrm{mean}\left(|\mathbf{Z}|\right). \tag{25}$$

Each transformed value was assigned to one of three levels:

$$T(Z_i) = \begin{cases} +1, & Z_i > \alpha/2, \\ 0, & |Z_i| \le \alpha/2, \\ -1, & Z_i < -\alpha/2. \end{cases} \tag{26}$$

The reconstructed weight tensor was

$$\widehat{\mathbf{W}} = \frac{1}{\sqrt{d}}\mathbf{H}^{\top}\left(\alpha T(\mathbf{Z})\right), \tag{27}$$

and activations were quantized to 8 bits using Eq. 18.

4 Experimental Setup

4.1 Models and Checkpoints

We evaluated two flow- or diffusion-based vision-language-action (VLA) models: $\pi_0.5$ and GR00T N1.7. For $\pi_0.5$, we used OpenPI's public `pi05_libero` checkpoint:

`gs://openpi-assets/checkpoints/pi05_libero`.

For GR00T, we used NVIDIA's suite-specific checkpoints from `nvidia/GR00T-N1.7-LIBERO`. Before evaluation, the GR00T runner verified that all checkpoint shards were loaded and rejected runs containing uninitialized model weights.

The models were executed through their native policy-server interfaces. $\pi_0.5$ used the OpenPI policy server and LIBERO client, while GR00T used NVIDIA's GR00T server and `rollout_policy.py`. This retained each model's standard preprocessing, action decoding, and simulator interface.

4.2 LIBERO Benchmark

We evaluated both models on all four LIBERO suites:

$$\mathcal{S} = \{\text{LIBERO-Spatial}, \text{LIBERO-Object}, \text{LIBERO-Goal}, \text{LIBERO-Long}\}. \tag{28}$$

Each suite contains ten language-conditioned manipulation tasks. Spatial tests spatial relationships, Object tests object-centric manipulation, Goal tests goal-conditioned behavior, and Long contains longer multi-stage tasks.

A trial or episode denotes one complete closed-loop rollout from environment initialization until task success, failure, or the evaluator's time limit. We use the terms *trial*, *episode*, and *rollout* interchangeably when referring to complete policy executions. Instrumented model calls used for tensor analysis are instead referred to as *diagnostic events*.

4.3 Rollout Protocol

For $\pi_0.5$, we evaluated 20 rollouts per task:

$$10 \text{ tasks} \times 20 \text{ rollouts} = 200 \text{ rollouts per suite}. \tag{29}$$

Each precision setting therefore included

$$4 \text{ suites} \times 200 \text{ rollouts} = 800 \text{ rollouts}. \tag{30}$$

The policy replanned after every five environment steps. Unlike GR00T, the OpenPI evaluator did not use or record an `n_envs` parameter; thus, five parallel environments should not be attributed to the $\pi_0.5$ experiments.

For GR00T N1.7, we evaluated 15 rollouts per task using five parallel environments:

$$10 \text{ tasks} \times 15 \text{ rollouts} = 150 \text{ rollouts per suite}, \tag{31}$$

or 600 rollouts per precision setting across the four suites. GR00T executed eight predicted action steps between policy queries, used four native denoising steps per action prediction, and allowed at most 720 environment steps per rollout.

All conditions used random seed 7. The same checkpoint, task ordering, action horizon, and evaluator configuration were retained across precision settings for each model.

4.4 Precision Configurations

We compared the following configurations:

$$\mathcal{P} = \{\text{FP16}, \text{A8}, \text{W4A8}, \text{W4A4}\}. \tag{32}$$

FP16 served as the full-precision baseline. A8 quantized activations to 8 bits while retaining floating-point weights. W4A8 used 4-bit weights and 8-bit activations, and W4A4 used 4-bit weights and activations.

The primary experiments used an action-specific quantization scope. For $\pi_0.5$, this covered the action-expert MLPs in all 18 action-expert transformer blocks. For GR00T, it covered the DiT feed-forward MLPs in all 12 DiT blocks. Attention projections, the vision-language backbone, and other non-action components remained in their original precision.

Tensors were rounded to the target numerical grid and reconstructed in their original floating-point type before matrix multiplication. Therefore, the experiments measure sensitivity to quantization noise rather than the memory and latency of packed integer inference.

4.5 Diagnostic Configuration

Before the full rollout matrix, we performed small LIBERO smoke tests to validate checkpoint loading, simulator execution, and tensor instrumentation. These runs used one task, one rollout, and one environment where applicable.

For K/V analysis, runtime hooks recorded unquantized and quantized tensors from a selected attention layer. Diagnostic collection was limited to 12 events, with at most 65,536 sampled values retained per tensor. The $\pi_0.5$ probe produced 12 indexed K/V records, while the GR00T probe produced four records corresponding to its four native denoising steps. Channel-statistics runs reduced tensors on the accelerator to per-channel RMS, maximum magnitude, and quantization mean-squared error instead of storing complete activations.

The exploratory K/V experiments used 3-bit WHT–Lloyd–Max quantization. A separate ITQ3_S-inspired W3A8 experiment applied transformed ternary weight reconstruction and 8-bit activation quantization. Neither exploratory setting was included in the final four-precision accuracy matrix.

4.6 Hardware and Execution

Experiments were executed as remote Modal jobs. $\pi_0.5$ ran on an NVIDIA A100 GPU with 40 GB of memory, while GR00T N1.7 ran on an NVIDIA H100. Since the two models used different GPU types, memory and latency values are compared across precision settings within each model, not directly between models.

Model checkpoints were cached in persistent storage, and every condition ran in a new evaluation process. The runner saved the server and client logs, per-task success rates, rollout counts, timing summaries, GPU samples, and experiment metadata after each completed condition.

4.7 Metrics and Logging

For N completed rollouts, success rate was computed as

$$\mathrm{SR} = \frac{1}{N} \sum_{i=1}^{N} 1 \left[\text{rollout } i \text{ succeeds}\right]. \tag{33}$$

The overall score for a precision setting was the unweighted mean of the four suite success rates.

GPU memory was sampled with `nvidia-smi` every two seconds. For rollout e, peak observed memory was

$$M_e^{\mathrm{peak}} = \max_{t \in e} M(t). \tag{34}$$

Memory was aggregated by task and precision configuration. Rollouts containing no GPU sample were marked as missing rather than assigned zero memory. This was particularly important for GR00T, where some short rollout intervals contained no sample.

5 Results

We evaluate the effect of activation and weight quantization on task success, GPU memory usage, and internal K/V representations. Unless otherwise stated, success rates are reported as percentages. The $\pi_0.5$ results contain 20 rollouts per task, or 800 rollouts per precision setting across all four LIBERO suites. The GR00T N1.7 results contain 15 rollouts per task, or 600 rollouts per setting.

5.1 Quantitative Analysis

5.1.1 LIBERO Task Success

Table 2 reports the $\pi_0.5$ success rates. The FP16 baseline achieved an average success rate of 97.63%. A8 retained 96.88%, corresponding to a reduction of only 0.75 percentage points. W4A8 achieved 95.88%, 1.75 points below FP16. In contrast, W4A4 reduced average success to 76.25%, a decrease of 21.38 points.

The largest $\pi_0.5$ W4A4 degradation occurred on LIBERO-Object, where success decreased from 99.00% to 61.00%. W4A4 also reduced LIBERO-Goal and LIBERO-Long performance to 76.50% and 73.00%, respectively. W4A8 was substantially more stable but still lost 4.50 points on LIBERO-Long. These results suggest that retaining 8-bit activations is important for preserving object-centric and long-horizon behavior.

Table 3 reports the corresponding results for GR00T N1.7. FP16 achieved an average success rate of 98.50%. A8 produced a measured average of 98.85%, while W4A8 retained 97.51%. Given the finite rollout sample, the 0.35-point A8 increase should be interpreted as matching FP16 rather than as evidence that quantization improves the model.

GR00T was highly robust to A8 and W4A8 on the Goal, Spatial, and Object suites, where these settings achieved 100% success or remained close to it. The principal difference appeared on LIBERO-Long. W4A8 reduced long-horizon success from 94.00% to 90.04%, while W4A4 reduced it to 42.67%. Therefore, aggressive activation quantization was substantially more damaging to multi-stage control than to the shorter suites.

Overall, A8 provided the most reliable accuracy-preserving setting. W4A8 also retained performance within two percentage points of FP16 for both models. W4A4 produced a qualitatively different regime, with large losses for both models and particularly severe degradation on long-horizon control.

5.1.2 GPU Memory Usage

Table 5 summarizes mean per-task peak GPU memory. The precision setting had effectively no influence on measured memory. Across presets, $\pi_0.5$ remained near 30.57 GB and GR00T remained near 7.16 GB. The mean preset-to-preset differences were at most a few megabytes, well below the task-dependent variation and the resolution required to claim meaningful compression.

This result is expected from our quantization implementation. The original floating-point weights remain resident in GPU memory, and quantized weights and activations are reconstructed in floating point before matrix multiplication. Consequently, these measurements do not estimate the memory savings of packed 4-bit parameters or low-bit kernels. Instead, they show that simulating low-bit arithmetic does not, by itself, reduce allocated GPU memory.

Figure 1 shows the task-level $\pi_0.5$ memory heatmap. Variation across tasks is visible, but each task has nearly identical memory usage under all four precision settings.

Figure 2 presents the GR00T task-level memory results. As with $\pi_0.5$, no systematic reduction is observed for A8, W4A8, or W4A4.

5.2 Qualitative and Representation Analysis

5.2.1 $\pi_0.5$ K/V Drift

Figure 3 compares unquantized and WHT–Lloyd–Max quantized $\pi_0.5$ K/V tensors. The quantized tensors retained a similar distribution to the unquantized tensors, and their late-step histograms largely overlapped. However, quantization slightly reduced RMS energy. The average relative RMS drift was approximately 1.75% for keys and 0.53% for values.

The measured live quantization MSE was approximately 0.009 for keys and 0.013 for values, substantially smaller than the static and independently recomputed quantization errors shown in the diagnostic plot. The unquantized $\pi_0.5$ K/V tensors also exhibited high kurtosis, approximately 98–100, indicating rare but substantial outliers. This heavy-tailed structure helps explain why activation precision and scale selection can strongly affect low-bit performance.

5.2.2 GR00T K/V Drift

Figure 4 shows the corresponding GR00T cross-attention analysis. Both the original and quantized RMS values remained constant across the four denoising steps, resulting in negligible temporal RMS drift. The quantized keys and values had lower RMS energy than their unquantized counterparts, and live quantization MSE was approximately 0.052 for keys and 0.051 for values.

Unlike the nearly continuous $\pi_0.5$ histograms, the GR00T quantized distributions concentrated at a small set of discrete reconstruction levels. This confirms that the quantizer was active and illustrates

Table 1: LIBERO evaluation protocol. Counts are reported per precision setting.

Configuration	$\pi_0.5$	GR00T N1.7
Suites	4	4
Tasks per suite	10	10
Rollouts per task	20	15
Rollouts per suite	200	150
Total rollouts	800	600
Parallel environments	Not specified	5
Action steps per query	Replan every 5 steps	8
Maximum episode steps	Suite dependent	720
Random seed	7	7

Table 2: $\pi_0.5$ success rate (%) across the four LIBERO suites. The Δ column reports the change in average success relative to FP16.

Precision	Goal	Spatial	Object	Long	Average	Δ
FP16	98.50	98.50	99.00	94.50	**97.63**	0.00
A8	95.50	98.50	98.00	**95.50**	96.88	-0.75
W4A8	98.00	98.00	97.50	90.00	95.88	-1.75
W4A4	76.50	94.50	61.00	73.00	76.25	-21.38

Table 3: GR00T N1.7 success rate (%) across the four LIBERO suites. The Δ column reports the change in average success relative to FP16.

Precision	Goal	Spatial	Object	Long	Average	Δ
FP16	100.00	100.00	100.00	94.00	98.50	0.00
A8	100.00	100.00	100.00	**95.42**	**98.85**	+0.35
W4A8	100.00	100.00	100.00	90.04	97.51	-0.99
W4A4	98.00	93.33	98.00	42.67	83.00	-15.50

Table 4: Average LIBERO success and degradation relative to FP16.

Model	Precision	Average (%)	Δ from FP16
$\pi_0.5$	FP16	97.63	0.00
$\pi_0.5$	A8	96.88	-0.75
$\pi_0.5$	W4A8	95.88	-1.75
$\pi_0.5$	W4A4	76.25	-21.38
GR00T N1.7	FP16	98.50	0.00
GR00T N1.7	A8	98.85	+0.35
GR00T N1.7	W4A8	97.51	-0.99
GR00T N1.7	W4A4	83.00	-15.50

Table 5: Mean peak GPU memory across LIBERO tasks. Minimum and maximum values show the range across tasks, not uncertainty across repeated runs.

Model	Precision	Mean (GB)	Minimum (GB)	Maximum (GB)
$\pi_0.5$	FP16	30.571	30.351	30.915
$\pi_0.5$	A8	30.574	30.354	30.919
$\pi_0.5$	W4A8	30.571	30.351	30.915
$\pi_0.5$	W4A4	30.573	30.351	30.915
GR00T N1.7	FP16	7.164	7.044	7.489
GR00T N1.7	A8	7.165	7.044	7.489
GR00T N1.7	W4A8	7.164	7.044	7.489
GR00T N1.7	W4A4	7.164	7.044	7.489

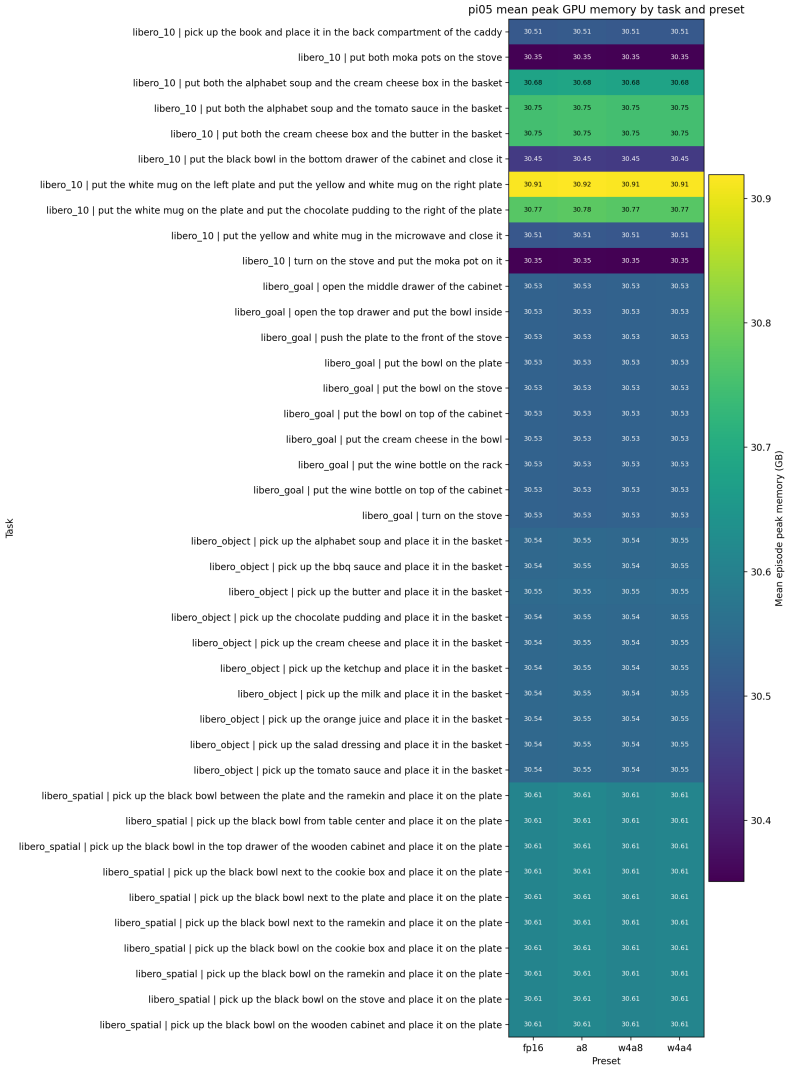


Figure 1: Mean episode peak GPU memory for $\pi_0.5$, grouped by LIBERO task and precision setting. Memory varies more across tasks than across precision settings.

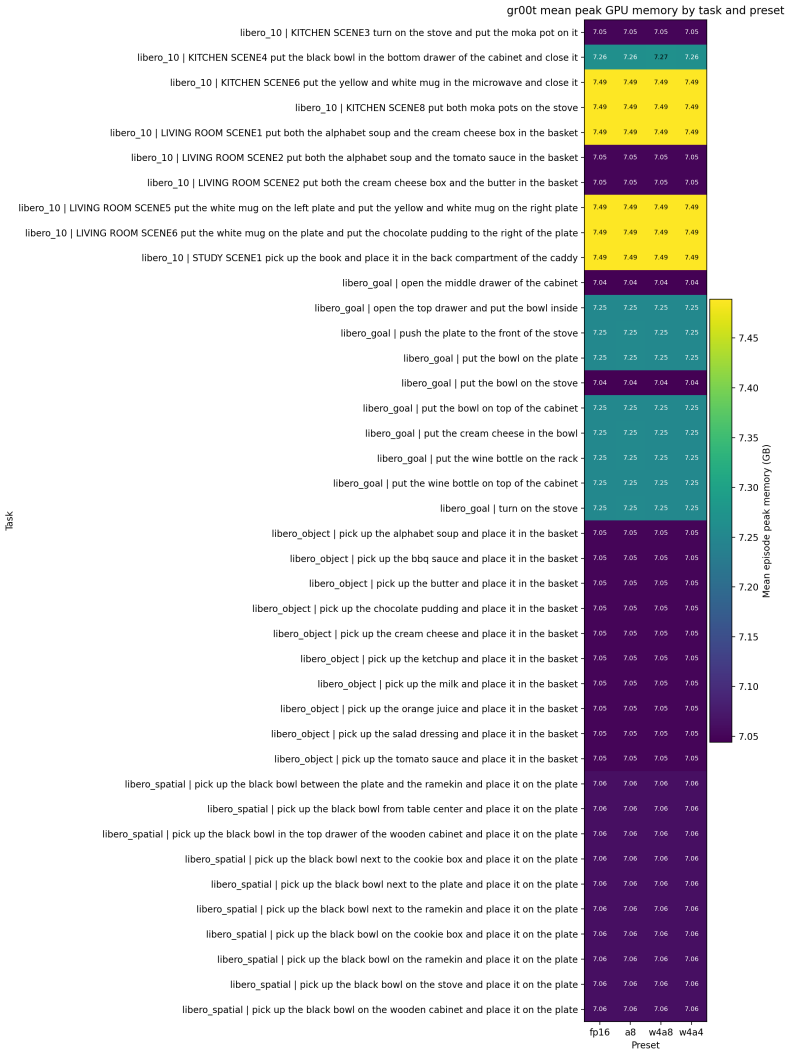


Figure 2: Mean episode peak GPU memory for GR00T N1.7, grouped by LIBERO task and precision setting.

the information loss hidden by aggregate RMS statistics. Nevertheless, the strong W4A8 and A8 rollout results indicate that GR00T tolerated moderate representation distortion when activation precision remained at 8 bits.

5.3 Summary of Findings

The experiments produce four principal findings. First, activation-only A8 preserved FP16 performance for both VLA models. Second, W4A8 introduced only small average losses, indicating that the action-generating MLP weights can tolerate four-bit perturbations when activations retain eight-bit precision. Third, W4A4 caused substantial failures, especially for object-centric and long-horizon tasks, showing that aggressive activation quantization is the dominant limitation in the evaluated implementation. Finally, our quantization approach did not reduce measured memory because it retained and operated on floating-point reconstructions.

The K/V diagnostics support the behavioral results but also show that no single tensor statistic fully predicts rollout success. Low live MSE and stable RMS can coexist with visibly discrete quantized distributions, while heavy-tailed activations may remain vulnerable to rare but consequential errors. VLA quantization should therefore be assessed using both internal representation diagnostics and closed-loop robotic rollouts.

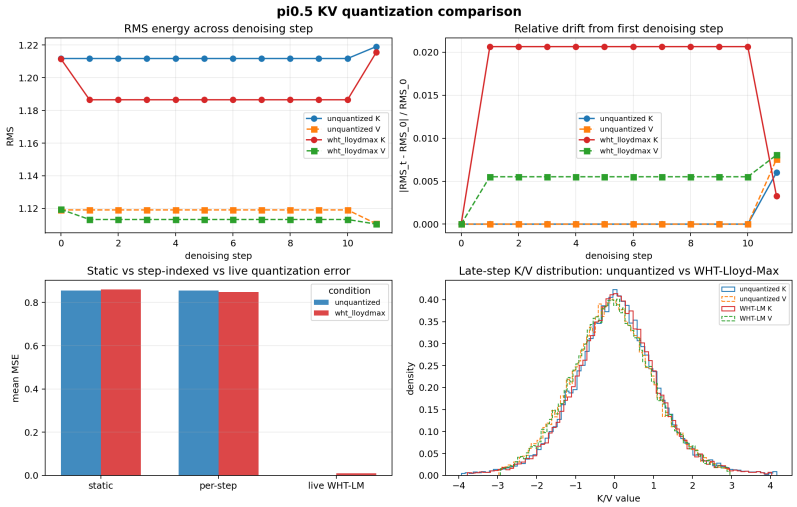


Figure 3: $\pi_0.5$ K/V comparison between the unquantized and 3-bit WHT–Lloyd–Max diagnostic conditions. The quantized distributions largely preserve the original shape, although key RMS energy shifts across denoising events.

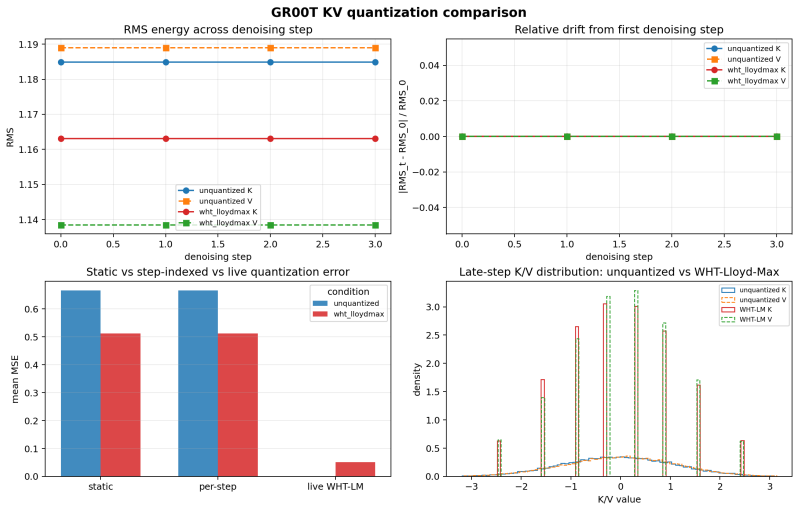


Figure 4: GR00T N1.7 cross-attention K/V comparison between unquantized and 3-bit WHT–Lloyd–Max diagnostic conditions. Temporal RMS drift is negligible, but the quantized histogram collapses onto discrete reconstruction levels.

6 Discussion

Our results indicate that moderate quantization is substantially more reliable than fully aggressive weight-activation quantization for the evaluated VLA policies. A8 remained within 0.75 percentage points of FP16 for $\pi_0.5$ and matched the GR00T N1.7 baseline within sampling variation. W4A8 also retained average performance within 1.75 and 0.99 points, respectively. In contrast, W4A4 reduced average success by 21.38 points for $\pi_0.5$ and 15.50 points for GR00T. The largest GR00T loss occurred on LIBERO-Long, suggesting that small action errors become more consequential when success requires a longer sequence of coordinated decisions.

On our LIBERO setup, GR00T N1.7 achieved higher raw success than the previously reported GR00T N1.5 QuantVLA results. However, this is not an apples-to-apples quantization improvement because the experiments use different model generations, checkpoints, evaluation protocols, and quantization implementations. Our result should instead be interpreted as evidence that fake-quantized A8 and W4A8 largely preserve the strong baseline accuracy of GR00T N1.7. It does not establish superiority over QuantVLA's compression method.

The K/V analyses provide a partial explanation for the observed behavior. Moderate quantization preserved RMS energy and produced relatively low live quantization error, while W4A4 provided insufficient activation resolution for the most sensitive tasks. The heavy-tailed $\pi_0.5$ distributions and the discrete GR00T 1.7 reconstructions also demonstrate that average MSE alone cannot fully describe policy robustness. Closed-loop rollouts remain necessary because infrequent tensor errors can alter object contacts or compound across a long-horizon trajectory.

Finally, the unchanged GPU-memory measurements are a limitation of the implementation rather than evidence that low-bit representations cannot compress VLA models. Our hooks perform fake quantization and reconstruct floating-point tensors before computation, leaving the original parameters allocated. The present study therefore evaluates numerical robustness, not deployment-level compression. Establishing practical memory and latency benefits will require packed low-bit weights, compatible quantized kernels at the Triton or Cuda level to see physical performance gains and reflect better accuracy, and evaluation on consistent hardware.

7 Conclusion

This work evaluated the quantization sensitivity of $\pi_0.5$ and GR00T N1.7 across four LIBERO task suites. A8 and W4A8 largely preserved FP16 success, whereas W4A4 caused substantial degradation, particularly on object-centric and long-horizon tasks. K/V drift and distribution analyses further showed that low average quantization error does not always guarantee closed-loop robustness. Because our implementation uses fake quantization, it evaluates numerical sensitivity without realizing low-bit memory compression. Due to the time limit of the quarter, future work would include using calibrated quantization, packed weights, and implementing hardware-supported kernels in Cuda for the various quantization modes, especially lower modes such as W4A4 or W3A8.

8 Team Contributions

- **William Yang** Did everything

Changes from Proposal The initial paradigm was investigating drift, and quantizing to W3A8. Preserving accuracy let alone improving it at W3 was very hard to do so we changed the proposal to that level.

References

X. Li et al. 2026. HBVLA: Hierarchical Binary Vision-Language-Action Models for Robotic Manipulation. *arXiv preprint arXiv:2602.13710* (2026).

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024a. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *Proceedings of the 7th MLSys Conference*.

Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2024b. QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving. *arXiv preprint arXiv:2405.04532* (2024). arXiv:2405.04532 [cs.CL] doi:10.48550/arXiv.2405.04532

Stuart P. Lloyd. 1982. Least squares quantization in PCM. *IEEE Transactions on Information Theory* 28, 2 (1982), 129–137. doi:10.1109/TIT.1982.1056489

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2022. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. *arXiv preprint arXiv:2211.10438* (2022). arXiv:2211.10438 [cs.CL] doi:10.48550/arXiv.2211.10438

Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. 2026. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. In *The Fourteenth International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2504.19874

Y. Zhang et al. 2026. QuantVLA: Efficient Vision-Language-Action Models via Post-Training Quantization. *arXiv preprint arXiv:2602.20309* (2026).